

Riding The Bus In Balatro: A Simulation Study

By: Evan Maurer
Topic # 13

ABSTRACT

This paper aimed to understand whether common Jokers in Balatro, the common Joker card known as “Ride The Bus,” and strategies optimized around it were sufficient for beating the game of Balatro, and whether other groups of common Jokers could outperform an RTB specific strategy. A variety of comparison tests were conducted on the data sets resulting from the distributions. Thorough analysis of each resulting simulation showed clear and significant differences. It was concluded that none of the simulated strategies could propel a player to Ante 8. However, an RTB driven strategy with the added use of common jokers showed better statistical outcomes than competing simulations. T

1. BACKGROUND: UNDERSTANDING BALATRO

In Balatro, a player is playing against a set number of points (chips) that belong to a given level (blind). In the base-state of the game – a player is given four hands to play in hopes of winning more chips than the base value attached to a blind. In Balatro “antes” act as the actual level. In each ante, a player typically faces three different blinds - the small blind, big blind, and boss blind each having different chip requirements. To beat the base-game, a player must have accumulated 558,450 chips to beat the 8 antes. Attached below are the base chip values, accessed from the games wiki, accessible by many contributors (Balatro Wiki Contributors, 2026)

Ante	Blind	Chip Requirement
Ante 1	Small Blind	300
	Big Blind	450
	Boss Blind	600
Ante 2	Small Blind	800
	Big Blind	1200
	Boss Blind	1600
Ante 3	Small Blind	2000
	Big Blind	3000
	Boss Blind	4000
Ante 4	Small Blind	5000
	Big Blind	7500
	Boss Blind	10000
Ante 5	Small Blind	11000
	Big Blind	16500
	Boss Blind	22000
Ante 6	Small Blind	20000
	Big Blind	30000

Ante 7	Boss Blind	40000
	Small Blind	35000
	Big Blind	52500
Ante 8	Boss Blind	70000
	Small Blind	50000
	Big Blind	75000
	Boss Blind	100000

Closer examination of this table would show that the accumulation of chips required to beat a blind appears to follow an exponential distribution across antes. The chip requirements at the surface level indicate a game design where players are faced with exponentially more difficult circumstances.

In Balatro players play hands common across other variants of Poker. In the attached picture, a player first playing the online version of the game are familiarized with the following scorecard.

lvl.1	Straight Flush	100 X8	# 0
lvl.1	Four of a Kind	60 X7	# 0
lvl.1	Full House	40 X4	# 0
lvl.1	Flush	35 X4	# 0
lvl.1	Straight	30 X4	# 0
lvl.1	Three of a Kind	30 X3	# 0
lvl.1	Two Pair	20 X2	# 0
lvl.1	Pair	10 X2	# 0
lvl.1	High Card	5 X1	# 0

Balatro. (Balatro, 2026)

Highlighted in blue are the base amount of chips a played hand automatically scores, highlighted in red is a multiplier applied to the total chip score of the hand once played. A played hand contains one to five cards by default.

Interestingly, developers have added mechanisms that allow players to supplement the base & multiplicative score of a hand. These mechanisms come in the form of special cards which exist separate from the deck & hand. Cards such as Jokers, Spectral, Planet, and Tarot Cards are all examples of these mechanisms. In Balatro, there are 150 unique Jokers that players can utilize (Balatro Wiki Contributors, 2026). Assuming none of these modifiers are active, the scoring mechanism in Balatro is explained as follows:

$$f(x) = \left(\text{base}(x) + \sum_{c \in P(x)} \text{rank}(c) \right) \cdot \text{mult}(x)$$

created with the help of Claude AI (Anthropic. (2026, April)

In this context, $P(x)$ refers to all the cards played in a particular hand, $rank(c)$ is each card's rank within $P(x)$, $base(x)$ denotes the base value assigned to the hand, and $mult(x)$ is the multiplier that corresponds to the specific type of hand being played. However, the mechanisms mentioned earlier are designed to serve an additive effect to this equation while certain conditions are met. Ride the Bus (RTB) is a common Joker randomly available for purchase from the games shop after beating a blind. Per Balatro's Wiki, RTB adds to base multiplier one additive multiplier for each consecutive hand played without a face card (Jack, Queen, or King) (Balatro Wiki Contributors, 2026). When a face card appears in a scored hand, the multiplier returns to zero. Referencing the prior scoring formula, a player who holds RTB and plays a hand without a scoring face card would score as follows.

$$f(x) = \left(base(x) + \sum_{c \in P(x)} rank(c) \right) \cdot (mult(x) + 1)$$

created with the help of Claude AI (Anthropic. (2026, April)

The hand multiplier effect of Ride The Bus is illustrated by a piecewise function.

$$Mult_{RTB}(n) = \begin{cases} 0, & \text{if hand } n \text{ contains a J, Q, or K,} \\ Mult_{RTB}(n - 1) + 1, & \text{otherwise.} \end{cases}$$

created with the help of Claude AI (Anthropic. (2026, April)

In other words, the multiplier resets to zero any time a face card is played in the hand but increases by one for each consecutive hand without a face card. Through the example of RTB, it is known that Jokers introduce optional choices of strategy. RTB encourages players to adopt an unfamiliar strategy. To help understand this, it is critical to understand how Balatro scores individual cards.

Balatro scores individual cards with the following base chips:

Card	Chips
Ace	11
King	10
Queen	10
Jack	10
10	10
9	9

8	8
7	7
6	6
5	5
4	4
3	3
2	2

In practice, RTB prevents players from benefiting from three high scoring cards – Jacks, Queens, and Kings, all worth ten points. In most other poker variants, high cards are usually seen as weak, but in RTB, this is reversed—a player might choose lower-value cards even in sacrifice of stronger hands if it helps them keep a multiplying advantage. This dichotomy raises the central question – how often does optimizing around a single strategy-defining joker like Ride the Bus produce statistically deeper runs than an unspecialized generalized strategy. To investigate this, a simulation was developed comparing RTB-optimized play against a generalized joker-involved strategy across randomly sampled selections of Jokers, using ante progression depth as the primary defining measure.

3. Methods

3.1. Pitfalls and Limitations of Simulating RTB

To simulate the scenarios of interest, understanding of the randomly generated effects of Balatro had to be better understood, but there are some limitations. These programmed simulations do not account for every single Joker, Boss type, Spectral Card, Tarot Card, Planet Card, or every possible deck. Rather, the simulation study aimed to compare RTB to Jokers of similar scope and commonality. RTB as a Joker is defined as a “common” Joker per Balatro’s wiki (Balatro Wiki Contributors, 2026). There are three general subsects of Jokers - Commons, Uncommons, and Rares. Available on the wiki were probabilities of said type occurring if a Joker were to appear for a player to buy- which are as follows.

$$P(X = x) = \begin{cases} 0.75 & \text{if } x = \text{common} \\ 0.25 & \text{if } x = \text{uncommon} \\ 0.05 & \text{if } x = \text{rare} \end{cases}$$

created with the help of Claude AI (Anthropic. (2026, April)

The simulation omitted rare and uncommon jokers from modeling to avoid further added variance. The use of spectral cards, tarot cards, and planet cards were not modeled to fully isolate the interactions between the programmed RTB and non-RTB generalized strategy with other common jokers. Also omitted were the effects of random bosses.

Bosses exist as the last barrier to beat an Ante. Though there are exceptions to this rule, bosses generally require the same effort in terms of chips but have harsher implications on strategy. Incorporating bosses in the program, as was attempted, proved to be difficult considering the amount of strategy implications that would have needed to be defined and accounted for. When considering how to model the impact of Jokers, specifically RTB, it was found that wide swaths of this game are simply too complex to incorporate given the scope of the research question. This paper generally includes Jokers which either added chips to the base score if certain conditions were met or multiplied the chip score if certain conditions were met. In total, 25 uncommon jokers, including RTB, were included in the simulation. However, this is not the full distribution of common jokers available in Balatro.

Balatro's complex economy system in which players can win in-game currency to buy jokers and other cards after winning a blind were also not modeled. Instead, the simulations assume that when Jokers are to be simulated, a player is limited to buy one joker per winning blind. Literature on the distribution of Jokers and their probability of them appearing appeared to be limited to probabilities associated with their rarities.

As the simulation focused on RTB as the primary card of interest, it was assumed that RTB was always the first card a player would see and buy when RTB was simulated with other Jokers. As is the condition in the base game, every run was only allowed to contain a maximum of 5 randomly selected Jokers, and only one could be purchased in between a blind. This is contrary to the shop system in Balatro where multiple Jokers can be available. It is not understood what the probability of Jokers appearing in a shopping slot were, only the distribution of possible outcomes if one were to occur were documented on the Wiki.

3.2.Simulating Jokers & Python's Random Module.

The Jokers included in the simulation either add chips to the base score or multiplier. To effectively program the simulation Jokers were divided into two groups which best explained their mechanism. This was to more easily model the scoring algorithm. The group which added to the multiplier contained sixteen Jokers, the group which added to a hand's base score contained eight Jokers.

When selecting a Joker, the simulation utilized Python's built in random module to generate pseudo-random numbers across a uniform distribution of [0, 23]. The probability of picking a Joker modeling the two groups were modeled as follows:

$$P(X = x) = \begin{cases} \text{add_chip_pool}(x), & x \in \{0, 1, 2, \dots, 7\} \\ \text{add_mult_pool}(x), & x \in \{8, 9, 10, \dots, 23\} \end{cases}$$

created with the help of Claude AI (Anthropic. (2026, April)

Once an integer was selected, a Joker was randomly selected with the random.choices function from the random module. This function returns a k=1 length sample from a given population of options (Python Software Foundation, 2026). A secondary method was created to keep track of Jokers and resample if a selected Joker was already being utilized by the player to avoid duplicate Jokers.

Use of random and its subsidiary functions were appropriate for the simulations as official documentation on the module indicates that random uses the Mersenne Twister as the core generator, which produces a 53-bit precision float and has a period length of $2^{19937}-1$ (Python Software Foundation, 2026).

Jokers did not become available for a player to use until after winning the first blind, which is standard across most game modes, especially for the Red Deck. As such, Jokers accumulate up to five, when toggled, after the player beats approximately five blinds.

3.3.Deck Creation & Limitations

The random module was also utilized in functions related to deck creation, drawing, and shuffling,

where its built-in shuffle function was used each time, a blind would start. Though there are many decks players can choose to play from in Balatro with many added effects, the simulation utilizes the Red Deck, Balatro's base deck representing a standard deck of cards. The deck uniquely grants players one extra discard - totaling four per blind.

3.4. Modeling the RTB Strategy.

Every simulation utilized a hand selection algorithm which guided the player on exactly what cards to pick. The player then has the option to play up to 5 cards per hand. The simulation assumes that the players are at least knowledgeable of poker and the scores of hands in Balatro.

The selection algorithm contained a boolean modifier representing the toggling of the RTB driven strategy. If any given simulation was to model the effect of RTB, a player would first discard any face cards in their hand and redraw from the deck up to four times per blind.

A discard is when a player opts to destroy any card in their hand, which will not be recycled back into the deck until blind is over and the deck is reset & shuffled. After the hand is empty of face cards or the player has run out of discards for said blind - the algorithm would then pre-score the hand and select the hand which would maximize the potential score.

It is acknowledged that the hand selection algorithm specifically enables an RTB focused strategy. What is aimed at being understood is just how different the results of an RTB-driven strategy is by comparing to three different simulations, with four simulations being conducted in total.

The default strategy modeled to compare against RTB does not utilize discards. This was a deliberate decision in order minimize the complexity associated with discarded decisions. For example, each Joker would require a strategy defining parameter. With up to five Jokers being able to exist at a given time, any attempt at simulating a parameter for every Joker would result in a computationally complex algorithm with high risk of failure and pitfalls where Jokers would have to be prioritized over one another. It is acknowledged that a random sample of strategies a player could

prioritize is possible, but the intense requirements of said method was of issue.

3.5. Scoring and Defining A win.

The scoring mechanism within the simulation is quite similar to the scoring functions introduced earlier. The model first selects a hand using by comparing the default score of every possible played hand within the combination of cards. To do this, the combinations function from the Itertools module in Python was utilized. This is true for both RTB and the default non RTB strategy.

Once the best possible hand was returned - the algorithm would then call on a list of jokers being used, if any, and apply the scoring conditions of each Joker to a separate pool of scoring modifiers. Using lambda functions, Jokers of interest were able to be passed to the scoring algorithm via dictionary, where conditionals were stored as values.

One pool represented the accumulated multipliers gained by playing the hand tied to one variable. Balatro multiplies the chip values after each card has been scored. Before multiplying, the chips gained from Jokers are also added to the base score. The modeled total multiplier can be defined with the equation below.

$$\text{Total_mult}(h) = a_{\text{mult}} + m_{\text{joker}} + c(h)$$

created with the help of Claude AI (Anthropic. (2026, April)

Where a_{mult} is defined as the added multipliers from the card RTB, where m_{Joker} is defined as the cumulative multiplier gained from meeting specific Joker conditions, and $c(h)$ is the base multiplier from given hand. These add up to give you a total multiplier to be applied to the hand. The same logic is applied for added chips. The full scoring logic is defined with the equation below

$$f(x) = \left(\text{base}(x) + \sum_{c \in P(x)} \text{rank}(c) + \sum_{j \in J(x)} \text{chips}(j, x) \right) \cdot \left(\text{mult}(x) + \sum_{j \in J(x)} \text{mult}(j, x) + \text{RTB}(x) \right)$$

created with the help of Claude AI (Anthropic. (2026, April)

Where $\text{mult}(j, x)$ is defined as:

$$\text{mult}(j, x) = \begin{cases} m_j(x), & \text{if condition of } j \text{ is satisfied in } x \\ 0, & \text{otherwise} \end{cases}$$

created with the help of Claude AI (Anthropic. (2026, April)

$RTB(x)$ is the summation of consecutive hands played where the played hand did not include a face card.

3.6.Simulation Types

Four simulations were conducted, with the primary unit of interest being the ante a run reached during the run of simulations. The four simulations, each containing 50,000 samples, were as follows.

- **Default:** No Jokers & no RTB effects were modeled, and Default hand selection method was used. This is effectively the control simulation.
- **RTB Only:** No additive Jokers and only RTB enabled. The hand selection strategy built for RTB was used.
- **RTB and Jokers:** RTB used in combination with up to four added Jokers. The RTB optimized strategy was used.
- **Jokers Only:** Up to 5 active Jokers were randomly sampled, and the default hand selection strategy was used.

3.7.Sampling

Each simulation contains a sample of 50,000 runs. In this simulation, a run is a game played by Balatro from start to finish. Recorded was each played hand, the value of the hand, the additive and multiplicative value of the jokers in each hand, information on discards, etc. Overall, a total of 200,000 runs across four simulations were completed during modeling.

To test whether enough sample was collected, each simulation and its outputs were tested for stationarity using the batch means process, where the data set was divided into $n = 10$ batches per simulation, where $\text{batch}(n) = 5000$ samples. This process tests whether the sample means are i.i.d normally distributed, indicating whether they are thereby stationary. A low variance across means indicates an adequate sample was collected. The results were as follows:

- **Default:** The batch means analysis showed a variance of 0.000 across all means, with a standard deviation of 0.0051, and a batch mean of 1.1325 antes reached.

- **RTB Only:** The batch means analysis showed a variance of 0.0004 across all batches with a Std. Dev of 0.0188, and a batch mean of 2.9639 antes reached.
- **RTB and Jokers:** The batch mean analysis showed a variance of 0.0003 across all means, an observed standard deviation of 0.0176, and a batch mean of 3.6246 antes reached.
- **Jokers Only:** The batch means analysis showed a variance of 0.004 across all means, a Standard Deviation of 0.0190, and a batch mean of 2.4562 antes reached.

The low variance across all sample means in each of the simulations shows that the threshold for convergence was met and an adequate sample was collected.

3.8. AI Usage.

Throughout the model making process, Claude was not specifically used to optimize or *fix any* functions of the code. Rather, it was used and fed errors to help debug occasionally difficult bugs. Early development of the code featured an incidental problem where a prior working file which was deleted as better methods became realized, was still incidentally being referenced in the long `game_state.py` code. Claude provided print statements to help debug certain problems given what was being seen. This was especially relevant when analyzing if models stored data in the expected distributions and format before analyzing them in a separate workbook. I was able to use them to debug parts of the code, but it proved rather unhelpful as I eventually figured out the problem. These print statements are commented out within the provided code.

Claude was used to help generate formulas shown throughout the paper as there did not appear to be sites out there where the user could create their own uniquely looking functions that molded well with the formatting of the paper. Claude was further used when plotting multiple distributions onto seaborn plots. When needed, Claude also was used to help guide efforts to only relevant sources, including that of documentation. In the analysis section of the program, SciPy was used in attempts to fit distributions to the resulting outputs, but no

statistically significant values indicated the distributions fitted any given simulation.

As such, Claude was also utilized to help better understand the programming technique in ensuring these tests were run properly. However, due to concern over accuracy, tests with use of SciPy were explicitly excluded from the results.

Claude was also utilized in helping add specific features to visualizations particularly within Seaborn generated plots. Claude was utilized to provide the toggle to ensure Distributions were to be displayed side by side instead of overlapping. Claude was asked to provide the toggle for this as the documentation proved to be unhelpful at times.

3.9. Research into Coding-Related Problems.

Some aspects of the code were inspired in part by publicly available domains. In the attached codebase, there are a variety of links when certain code was inspired by information obtained from said link. More specifically, the use of the combinations () function from intertools was partly obtained from this resource, where a solution most matching the problem, I was trying to solve was available (CodingTechRoom, n.d.).

Further examples of using public resources to help solve certain programming problems come from the initial deck construction. More specifically, figuring out the optimal way of storing a card within a deck and hand was of question, and Stack Overflow proved as a valuable resource in accelerating me past that block (Stack Overflow Contributors, 2019).

4. Findings

4.1. The Kruskal-Wallis H-test.

To understand differences between underlying distributions, The Kruskal Wallis H Test for comparing distributions was conducted, compared to RTB only. This method tests the null hypothesis that the median population of all the groups are equal (SciPy developers, 2026b). However, it is also noted in documentation that rejecting the null hypothesis is not full proof evidence the distributions are different and further testing is necessary.

When comparing all 4 distributions, the test showed an extremely low P-value of < 0.001 with an H-statistic of 79,270.893. This suggests that the distributions could be different but may not be an unusual outcome given the scale of the datasets.

1. Chi Square Test for association between three simulated distributions.

Following the results of the Kruskal Wallis H-test, a chi squared test was conducted on three of the four distributions, with the default simulation being excluded.

The default simulation was excluded because the distribution was not compared because of the amount of zero observations existing past ante two, breaking the program when attempted to be run.

When comparing the three test cases, the outcomes were surprising. Using scipys chi2_contingency function, the p value was reported as $p < 0.001$ was recorded at 10 degrees of freedom with a Chi-Square statistic of 37,781.973, providing further evidence that there are significant differences between the distributions compared to RTB Only.

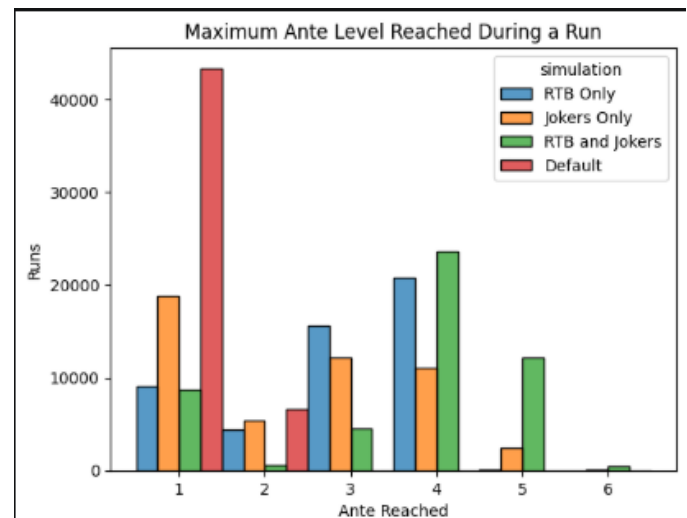


Figure 1, created with code provided in part by Claude AI (Anthropic, (2026, April)

Figure one shows each simulation that fared across antes in terms of the amount of runs which extended into a given ante. Notably, the steep cut off at ante one within the default simulation shows low chance of survival through playing the most likely hands alone. This suggests that even when making the best possible hand decisions, ante one is still progressively difficult without getting lucky with

high scoring hands, or without the added effect of Jokers.

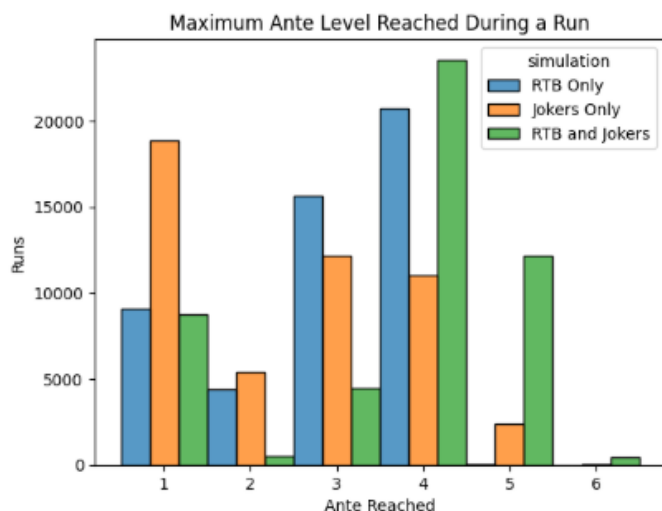


Figure 2, created with code provided *in part* by Claude AI (Anthropic. (2026, April)

Figure 2 shows the three experimental simulations and their distributions compared to one another. This visual further supports a high degree of difference between the three sets of data. Further supporting the findings of the Chi test. Many interesting observations can already be taken from these distribution plots. However, instead of assuming implicit differences, further testing between the distributions was conducted.

4.2.Mann-Whitney U Rank tests.

In the SciPy module, many other test statistics are available to use. For the purposes of deciding which distributions are greater or less than one another comparatively, the Mann-Whitney U test appeared to be the best available. It is a nonparametric test of the null hypothesis that the distribution in the underlying sample X is the same as the distribution in the underlying sample Y (SciPy developers, 2026c). This test was conducted across the three sets of distributions, and each result indicated a P-value of $P < 0.001$, indicating that we are to reject the null hypothesis that the distributions are the same among any pair.

Further use of this method of comparison shows that there is a particular ranking of overall distributions that maximize potential ante.

1. When comparing an RTB only strategy where the RTB card alone is played after winning blind $n = 1$, the results show that with a P-value of $P < 0.001$, the

RTB only strategy featured a distribution of antes reached that is greater than the distribution of the generalized Joker strategy.

2. When combining other Jokers with RTB, the results show that with a P-value of $P < 0.001$, RTB and Jokers produce statistically deeper runs than a generalized Joker Strategy Alone.
3. When utilizing strategy using the RTB card alone, results show a P-value of <0.001 , indicating that an RTB alone strategy is not sufficient in beating a strategy utilizing added jokers on top of an RTB card, however random the subset of added Jokers may be.

4.3. Confidence Intervals

The use of a boot strap method for conducting confidence intervals was conducted. However, for more finite precision the use of blinds played as a proxy for was used. Blinds are contained within antes and represent a very similar distribution to antes. There are generally three blinds played per ante. The results are as follows.

1. Default: [1.1295, 1.1354]
2. RTB only [7.7214, 7.836]
3. Jokers Only [6.2391, 6.3115]
4. RTB and Jokers [9.8062, 9.8835]

When looking at the rounds played level, the confidence intervals do not appear to overlap across all four distributions, providing certainty that they are each uniquely different from one another. If analyzed on the ante level, results could be misleading in the case of confidence intervals.

Evidence provided by the Mann Whitney tests show that an RTB driven strategy does in fact result in deeper runs than a generalized joker strategy, let alone a default strategy not utilizing jokers which was skewed far to the left of other simulated distributions. Accordingly, the null hypothesis of penetration into further antes was rejected as there is a statistically significant advantage to prioritizing the use of the RTB Jokers.

4. Discussion & Areas For Future Research

Referring Figure 1, it is observed that Generalized Joker Strategies and Default Strategies often logjam at ante ante one.

Ante	Default	RTB Only	Jokers Only	RTB and Jokers
Ante 1	0.8675	0.1822	0.3776	0.1753
Ante 2	0.1325	0.0884	0.1088	0.0109
Ante 3	0.0000	0.3133	0.2437	0.0895
Ante 4	0.0000	0.4155	0.2210	0.4715
Ante 5	0.0000	0.0006	0.0476	0.2439
Ante 6	0.0000	0.0000	0.0013	0.0089
Ante 7	0.0000	0.0000	0.0000	0.0000
Ante 8	0.0000	0.0000	0.0000	0.0000

Figure 3

Closer examinations of the probabilities of getting to each ante across the simulated types show an interesting trend. The RTB card was able to allow players to have the best chance of reaching Ante three and four across the tested strategies, but with RTB and Jokers being utilized, 25.19% of runs did reach ante five.

Though the Manns Whitney U- Test confirmed that the distribution of ante reached within the RTB simulation was greater than that of a generalized Joker strategy, the generalized Joker Strategy had more observations that resulted in the Joker reaching Ante 6, whilst RTB was noted to not have a single observation reaching ante six. Although the RTB driven strategy provides a better cumulative probability of reaching deeper antes than the prior, the prior still had runs that got deeper, but the overall distribution still skewed left of the RTB only strategy.

With no strategy being defined for Jokers only, this is evidence that there is at least some merit in combining a random assortment of jokers with a

normal hand strategy that could result in a deeper run. The odds, however, still indicate extreme difficulty when RTB and Jokers can be used altogether.

Furthermore, it appears that although the RTB added Joker strategy appears to have the best results, they still didn't allow for deeper penetration into Ante six, but most definitely give players better odds of thriving in Antes four and five.

Further deep dives into the data will show clear reason as to why a RTB based generalized Joker strategy is preferential to default, or a generalized joker strategy.

4.1. Interesting Deviations

From referencing the previous images, it was shown that players had better outcomes with an RTB focused strategy utilizing random subsets of Jokers.

Average Multiplier Per Ante			
Ante	RTB Only	Jokers Only	RTB and Jokers
Ante 1	0.2131	2.3588	2.0960
Ante 2	3.6568	5.1026	6.7049
Ante 3	9.5916	10.2115	13.4982
Ante 4	14.1467	13.9259	20.7568
Ante 5	19.3424	16.2578	24.1705
Ante 6	NaN	24.6122	28.1066

Figure 4

The findings indicate that, for the initial three Antes, employing a randomized joker strategy resulted in higher average multipliers compared to using the RTB-only strategy. Additionally, each simulation demonstrated that the average multiplier utilized per round increased at a comparable rate across all rounds. On average, players can anticipate gaining +3.4437 total multiplier per ante per hand with the RTB Only strategy, +4.4507 multiplier per ante per hand with the generalized Joker Strategy, and +5.2021 multiplier per ante per hand when utilizing both the RTB and Jokers strategy.

It was expected initially that the distribution of said multiplier values would be uniform after ante two. This was clearly not the case, and further analysis into the hand decisions of each strategy may shed some light. The multipliers due to hand type was excluded from the dataset in the table and are not considered to be supplementing the score. As such, players playing better hands in deeper Antes is not a

sensible explanation. This unexpected phenomenon serves as an area of further research or could be a sign of an underlying problem in the framework of the simulation.

More interestingly, when examining the distribution of average chips scored per hand per run per simulation, it starts to become even more evident how a RTB maximizing and randomly simulated joker combination performs better in this simulation. Although a clear distinguishable difference between these distributions in terms of antes reached has been statistically proven as significant, the degree of which may be even more explainable by the average chip score per run.

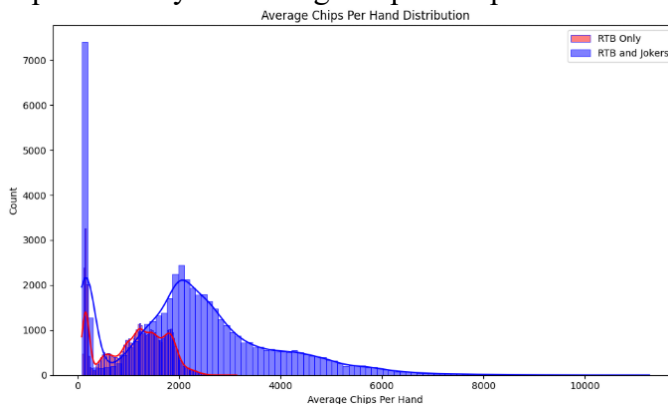


Figure 5, created with the help of Claude AI (Anthropic. (2026, April))

What is shocking is that despite the distribution of chips clearly indicating significant scoring advantages for RTB and Joker simulations, the massive benefit of added chips could not propel any run past Ante 6. Which serves as further evidence that Balatro is designed to be an exponentially more difficult game.

It is notable that examination of the visual distributions of the played hands did not yield significant variation across most hands between any of the simulations.

4.2. Areas of Further Research & Further Limitation

As described in the methods section, many areas significant to the real games function were avoided altogether. Areas of future research would include further specific analysis on how each type of Joker interacts with one another throughout a simulation. More specifically, whether there is a most optimal combination of Jokers is a question that Balatro players have long wanted an answer for. Though this paper doesn't examine that question, nor come close to help shed light on it – What the paper did do was randomly simulate 25 actual Jokers from

Balatro. However, whether each Joker combination obtained enough sample to analyze their interactions was not tested. As such, information pertaining to joker distributions was not fully explored. It is notable that when looking at the Jokers and RTB only distributions in terms of jokers - Jokers such as Banner, Crafty Joker, Misprint, Odd Todd, and RTB together contained the highest average chips gained per hand of 6,0842.10.

These were common Jokers which provided either additive multiplication bonuses, or additive chip bonuses. The simulation of these jokers suggests a combination of a generalized RTB approach should be utilized. However, even with this strategy the simulation never got past Ante 6. Moreover, even when using an optimal discard strategy under RTB only simulations, no run got past Ante 5. This shows that there may be a potential limit to what common jokers can propel a player to ante-wise, and further incorporation and analysis of common jokers is necessary. Further, research on how many additional rare or uncommon jokers would be necessary to propel a player to the next antes could provide interesting insight. Furthermore, analysis on the limits of RTB versus specific bosses could be an area for future analysis.

The reader may notice that throughout the paper, no formal name was given to the distribution of antes across the simulations. This was not out of lack of foresight. Multiple fitting algorithms and research to find ways to best assess the distribution of the data was conducted, but said research was timely and inconclusive on the best method to do so in Python.

5. Conclusion

This paper sets out to determine whether a RTB maximizing strategy could prove to be statistically better than a generalized joker strategy and default strategies in Balatro. Through a variety of statistical tests, it was determined that RTB prioritizing strategies in combination with commonly seen jokers do provide a maximizing potential greater than the other simulated strategies. Interestingly, the simulation also showed that despite there being 8 antes in Balatro, not a single player in the simulations involving any combination of common jokers or RTB were able to get past ante six. This serves as further evidence of Balatro's exponential

difficulty, and that common cards alone are not sufficient in completing a game of Balatro.

Bibliography:

Anthropic. (2026). *Claude Sonnet 4.6* [Large language model]. <https://claude.ai>

Balatro Wiki Contributors. (2026). *Balatro Wiki*. <https://balatrowiki.org>

CodingTechRoom. (n.d. accessed 2026). *How to determine if a poker hand is a straight?* <https://codingtechroom.com/question/-determine-poker-hand-straight>

LocalThunk. (2024). *Balatro* [Video game]. Playstack. <https://store.steampowered.com/app/2379780/Balatro/>

PythonGuides. (Kumar, 2025). *Scipy confidence interval*. PythonGuides. <https://pythonguides.com/scipy-confidence-interval/>

Python Software Foundation. (2026a). *random — Generate pseudo-random numbers* (Python 3.14.4 documentation). <https://docs.python.org/3/library/random.html>

Python Software Foundation. (2026). *dataclasses — Data Classes* (Python 3.14.4 documentation). <https://docs.python.org/3/library/dataclasses.html>

SciPy developers. (2026). *scipy.stats.kruskal* (SciPy v1.17.0 Manual). <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kruskal.html>

SciPy developers. (2026). *scipy.stats.mannwhitneyu* (SciPy v1.17.0 Manual). <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html>

SciPy developers. (2026). *scipy.stats.chi2_contingency* (SciPy v1.17.0 Manual). https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chi2_contingency.html

Stack Overflow Contributors. (2019). *Identifying straight, flush and other categories (from Poker) using Python*. Stack Overflow. <https://stackoverflow.com/questions/56874468/identifying-straight-flush-and-other-categories-from-poker-using-python>